CSC 478 – Project Plan

Log Analyzer & Report Generator (LARG)

Team Members – Group 6: Amanda, Mark, Lian and Lian

Technology Stack: Python 3.x, PyQt, Windows 10, HTML & CSV exports

1.0 Project Overview

The Log Analyzer & Report Generator (LARG) is a standalone desktop application designed to help users analyze log files efficiently. The application will allow users to import one or more log files, parse their contents, search and filter entries, identify recurring patterns or errors, and export analysis results into structured reports.

2.0 Scope Statement

The Log Analyzer & Report Generator (LARG) is a standalone desktop application that enables users to import, parse, analyze, and report on log files.

Project Title: Log Analyzer & Report Generator (LARG)

2.1 Purpose

As we know, Log files contain valuable operational information, but manually reviewing large logs is time consuming and error prone. Therefore, this project will build a Windows 10 desktop tool that ingests one or more log files, allows users to search or filter, produces summaries and statistics, flags important patterns, and exports a report for troubleshooting and documentation.

2.2 Platform Target OS

Windows 10 (design and testing will be done for Windows 10).

2.3 Standalone vs Network

- The software will function as a standalone desktop application on a single computer.
- A network connection is not required for core functionality.

2.4 Interaction with Other Software

- The software will read local log files (plain text) selected by the user.
- The software will export results like CSV and HTML files saved locally.
- The software will not require the instructor/evaluator to install any server-side framework or services.

2.5 Programming Language and Tooling

- Python 3.x

- Packaging: PyInstaller to produce a Windows-ready executable installer.

2.6 User Interface

- The software will use a graphical user interface (GUI) built with PyQt.
- The application will support file selection (file/folder), search/filter controls, results display, summary view, and export buttons.

3.0 Organization and Roles

- David – Team Lead - Analytics and Reporting
  - Project coordination, scheduling, integration
  - Milestone reports and final submission
  - Summary statistics and pattern detection
  - HTML and CSV report generation
- Amanda – User Interface
  - PyQt UI design and implementation
  - UX consistency and usability
- Mark – User Workflow, Parsing and Searching
  - User workflow (Import → Analyze → Export)
  - File ingestion and parsing logic
  - Search, filtering, and sorting mechanisms
  - Support for structured and generic parsing modes
- Lian – Testing and Packaging
  - Test planning, test execution, and packaging
  - Repository management and version control

4.0 Development Approach

The project will follow an iterative and incremental development model, allowing features to be built, tested, and refined in stages.

Planned Iterations

- Iteration 1 (MVP):
- File import
- Generic log parsing
- Basic search
- CSV/HTML export

Iteration 2:

- Structured parsing (timestamp level message)
- Severity and time-based filtering
- Summary statistics

Iteration 3:

- Pattern detection and rule-based flagging

- Enhanced reports
- UI polish

Iteration 4 (Stabilization):

- Performance tuning
- Packaging and installation testing
- Documentation finalization

5.0 Schedule

See detailed schedule table below.

6.0 Tools and Standards

- **Coding Standards:** PEP 8, meaningful naming, inline documentation
- **Testing:** Unit tests, integration tests, regression tests
- **Documentation:** Clear requirements, design diagrams, user manual
- **Process:** Iterative development with frequent integration

7.0 Configuration Management - proposed

- Version Control: GitHub repository
- Branching Strategy:
  - *main* – stable releases
  - *develop* – integration branch
  - *feature/** – feature-specific work
- Versioning: Semantic versioning (e.g., 1.0.0)
- Release Management: Always maintain at least one known-good build

8.0 Communication Plan

- Primary communication via Microsoft Teams
- Weekly team check-ins

9.0 Risks and Mitigation

- **Log format variability:** Use fallback generic parsing
- **Performance with large files:** Use efficient parsing and pagination
- **Packaging issues:** Early and repeated installation testing
- **Schedule risk:** Iterative milestones and frequent integration.

10.0 Success Criteria

The project will be considered successful if:

- The application installs and runs correctly on Windows 10
- Core features (import, parse, search, analyze, export) function as specified

Detailed Schedule Table – Gantt Chart

PROJECT: Log Analyzer & Report Generator (LARG)

Group 6

Legend: On track | Low risk | Med risk | High risk | Not Started

Project start date: 2/1/2026

Scrolling increment: 16

Milestone description	Category	Assigned to	Progress	Start	Days
Kickoff and Scope Finalization	Milestone				
Team assembly	On Track	Instructor	100%	2/1/2026	5
Scope Development	On Track	David	100%	2/6/2026	5
Workflow Development	On Track	Mark	100%	2/11/2026	10
Project Plan	On Track	David	100%	2/23/2026	22
Review of the Project Plan	On Track	Everyone	100%	3/1/2026	6
Design	Milestone		100%		
UI design	On Track	Amanda	100%	3/29/2026	28
Parsing design	On Track	Mark	100%	3/29/2026	28
Analytics and reports	On Track	David	100%	4/12/2026	14
Core implementation	On Track	Amanda, Mark and David	100%	4/19/2026	7
Testing	Milestone		100%		
Testing and integration	On Track	Lian	100%	4/26/2026	7
Packaging and documentation	On Track	Lian	100%	5/3/2026	7